

▼ Phần 1: Làm quen với Pytorch

```
import torch
import numpy as np
# Tạo tensor từ list
data = [[1, 2], [3, 4]]
x_data = torch.tensor(data)
print(f"Tensor từ list:\n {x_data}\n")
# Tạo tensor từ NumPy array
np_array = np.array(data)
x_np = torch.from_numpy(np_array)
print(f"Tensor từ NumPy array:\n {x_np}\n")
# Tạo tensor với các giá trị ngẫu nhiên hoặc hằng số
x_ones = torch.ones_like(x_data) # tạo tensor gồm các số 1 có cùng shape với x_data
print(f"Ones Tensor:\n {x_ones}\n")
x_rand = torch.rand_like(x_data, dtype=torch.float) # tạo tensor ngẫu nhiên
print(f"Random Tensor:\n {x_rand}\n")
# In ra shape, dtype, và device của tensor
print(f"Shape của tensor: {x_rand.shape}")
print(f"Datatype của tensor: {x_rand.dtype}")
print(f"Device lưu trữ tensor: {x_rand.device}")
```

Tensor từ list:
tensor([[1, 2],
 [3, 4]])

Tensor từ NumPy array:
tensor([[1, 2],
 [3, 4]])

Ones Tensor:
tensor([[1, 1],
 [1, 1]])

Random Tensor:
tensor([[0.2594, 0.6041],
 [0.0860, 0.0022]])

Shape của tensor: torch.Size([2, 2])
Datatype của tensor: torch.float32
Device lưu trữ tensor: cpu

x_data + x_data

tensor([[2, 4],
 [6, 8]])

x_data*5

tensor([[5, 10],
 [15, 20]])

x_data*x_data.T

tensor([[1, 6],
 [6, 16]])

x_data @ x_data.T

tensor([[5, 11],
 [11, 25]])

x_data[0,:]

tensor([1, 2])

x_data[:,1]

tensor([2, 4])

x_data[1,1]

tensor(4)

x_rand = torch.rand(4,4)

x_rand.view(16,1)

```
tensor([[8.7124e-01],
        [5.7832e-01],
        [3.8449e-01],
        [7.1092e-01],
        [4.2942e-01],
        [1.3047e-01],
        [7.1220e-01],
        [8.8841e-04],
        [4.9977e-03],
        [5.3957e-01],
        [4.5803e-01],
        [4.2540e-01],
        [9.6446e-01],
        [3.6032e-01],
        [5.2042e-01],
        [9.3121e-01]])
```

```
x_rand.reshape(16,1)
```

```
tensor([[8.7124e-01],
        [5.7832e-01],
        [3.8449e-01],
        [7.1092e-01],
        [4.2942e-01],
        [1.3047e-01],
        [7.1220e-01],
        [8.8841e-04],
        [4.9977e-03],
        [5.3957e-01],
        [4.5803e-01],
        [4.2540e-01],
        [9.6446e-01],
        [3.6032e-01],
        [5.2042e-01],
        [9.3121e-01]])
```

▼ Phần 2: Tự động tính đạo hàm với autograd

```
# Tạo một tensor và yêu cầu tính đạo hàm cho nó
x = torch.ones(1, requires_grad=True)
print(f"x: {x}")
# Thực hiện một phép toán
y = x + 2
print(f"y: {y}")
# y được tạo ra từ một phép toán có x, nên nó cũng có grad_fn
print(f"grad_fn của y: {y.grad_fn}")
# Thực hiện thêm các phép toán
z = y * y * 3
# Tính đạo hàm của z theo x
z.backward() # tương đương z.backward(torch.tensor(1.))
# Đạo hàm được lưu trong thuộc tính .grad
# Ta có z = 3 * (x+2)^2 => dz/dx = 6 * (x+2). Với x=1, dz/dx = 18
print(f"Đạo hàm của z theo x: {x.grad}")
```

```
x: tensor([1.], requires_grad=True)
y: tensor([3.], grad_fn=<AddBackward0>)
grad_fn của y: <AddBackward0 object at 0x7ccc3a8c07f0>
Đạo hàm của z theo x: tensor([18.])
```

```
#z.backward()
```

Khi gọi `z.backward()` một lần nữa sẽ bị lỗi vì sau khi gọi `backward` lần đầu thì Pytorch đã giải phóng toàn bộ đồ thị tính toán để tiết kiệm bộ nhớ

▼ Phần 3. Xây dựng mô hình đầu tiên với torch.nn

```
# Khởi tạo một lớp Linear biến đổi từ 5 chiều -> 2 chiều
linear_layer = torch.nn.Linear(in_features=5, out_features=2)
# Tạo một tensor đầu vào mẫu
input_tensor = torch.randn(3, 5) # 3 mẫu, mỗi mẫu 5 chiều
# Truyền đầu vào qua lớp linear
output = linear_layer(input_tensor)
print(f"Input shape: {input_tensor.shape}")
print(f"Output shape: {output.shape}")
print(f"Output:\n {output}")
```

```
Input shape: torch.Size([3, 5])
Output shape: torch.Size([3, 2])
Output:
tensor([[ 0.1646,  0.5173],
```

```
[-0.1512, -0.5032],  
[ 0.2802, -0.4078]], grad_fn=<AddmmBackward0>)
```

```
# Khởi tạo lớp Embedding cho một từ điển 10 từ, mỗi từ biểu diễn bằng vector 3 chiều  
embedding_layer = torch.nn.Embedding(num_embeddings=10, embedding_dim=3)  
# Tạo một tensor đầu vào chứa các chỉ số của từ (ví dụ: một câu)  
# Các chỉ số phải nhỏ hơn 10  
input_indices = torch.LongTensor([1, 5, 0, 8])  
# Lấy ra các vector embedding tương ứng  
embeddings = embedding_layer(input_indices)  
print(f"Input shape: {input_indices.shape}")  
print(f"Output shape: {embeddings.shape}")  
print(f"Embeddings:\n {embeddings}")
```

```
Input shape: torch.Size([4])  
Output shape: torch.Size([4, 3])  
Embeddings:  
tensor([[ 0.4482,  0.4121, -0.6997],  
        [-0.6387,  0.9303, -0.9023],  
        [-2.0666, -1.0410, -2.1688],  
        [ 0.9114, -3.3607, -0.1660]], grad_fn=<EmbeddingBackward0>)
```

```
from torch import nn  
class MyFirstModel(nn.Module):  
    def __init__(self, vocab_size, embedding_dim, hidden_dim, output_dim):  
        super(MyFirstModel, self).__init__()  
  
        self.embedding = nn.Embedding(vocab_size, embedding_dim)  
        self.linear = nn.Linear(embedding_dim, hidden_dim)  
        self.activation = nn.ReLU()  
        self.output_layer = nn.Linear(hidden_dim, output_dim)  
  
    def forward(self, indices):  
        embeds = self.embedding(indices)  
        hidden = self.activation(self.linear(embeds))  
        output = self.output_layer(hidden)  
        return output
```

```
# Khởi tạo và kiểm tra mô hình  
model = MyFirstModel(vocab_size=100, embedding_dim=16, hidden_dim=8, output_dim=2)  
input_data = torch.LongTensor([[1, 2, 5, 9]]) # một câu gồm 4 từ  
output_data = model(input_data)  
print(f"Model output shape: {output_data.shape}")
```

```
Model output shape: torch.Size([1, 4, 2])
```